

ITA 0608 - MACHINE LEARNING

3 . Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim

To build an Artificial Neural Network (ANN) using the Backpropagation algorithm, train it with an appropriate dataset, and evaluate its performance by testing it on unseen data for accurate classification or prediction.

Algorithm

- Initialize the neural network with input, hidden, and output layers.
- Assign random weights and biases to all connections.
- Input the training dataset to the network.
- Perform forward propagation:
 - Calculate the weighted sum of inputs.
 - Apply an activation function (e.g., Sigmoid or ReLU).
 - Generate the output.
- Compute the error by comparing the predicted output with the target output.
- Calculate the output layer error using the derivative of the activation function.
- Backpropagate the error to the hidden layer(s).
- Update the weights and biases using gradient descent:
 - $\text{New Weight} = \text{Old Weight} + (\text{Learning Rate} \times \text{Error} \times \text{Input})$
- Repeat Steps 3–8 for all training samples and continue for multiple epochs until the error reaches a minimum or the maximum number of epochs is completed.
- Test the trained network using a separate test dataset.
- Evaluate the performance using accuracy or other performance metrics.
- End.

Code

- **Weight and Bias Initialization:** Random small values.

```
def initialize_parameters(input_size, hidden_size, output_size):
    np.random.seed(42)
    weights_input_hidden = np.random.randn(input_size, hidden_size) * 0.01
    weights_hidden_output = np.random.randn(hidden_size, output_size) * 0.01
    bias_hidden = np.zeros((1, hidden_size))
    bias_output = np.zeros((1, output_size))
    parameters = {
        'W1': weights_input_hidden,
        'b1': bias_hidden,
        'W2': weights_hidden_output,
        'b2': bias_output
    }
    return parameters

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)

input_size = X_data_scaled.shape[1]
hidden_size = 4
output_size = 1
parameters = initialize_parameters(input_size, hidden_size, output_size)
print("\n--- Initialized Parameters ---")
print("W1 shape:", parameters['W1'].shape)
print("b1 shape:", parameters['b1'].shape)
print("W2 shape:", parameters['W2'].shape)
print("b2 shape:", parameters['b2'].shape)
```

Output

```
...
--- Initialized Parameters ---
W1 shape: (4, 4)
b1 shape: (1, 4)
W2 shape: (4, 1)
b2 shape: (1, 1)
```

Result

An **Artificial Neural Network (ANN)** was successfully built and trained using the **Backpropagation algorithm**. The network learned the patterns from the training dataset by iteratively updating the weights to minimize the error. The trained model was tested with an appropriate dataset and produced accurate predictions, demonstrating the effectiveness of the Backpropagation algorithm for classification/prediction tasks.